COS30018 - Intelligent System

Project: Stock Price Prediction System

Task C.1 - Project Environment Setup

Student Name: Ngo Sy Vuong
Student ID: 105551480
Tutor: Nguyen Manh Toan
Tutorial Session: Wednesday 8:00 - 12:00
Semester: May - July 2026

SWINBURNE UNIVERSITY OF TECHNOLOGY

Table of Contents

Environment Setup

To successfully replicate and run the stock price prediction model, please fulfill the following structural and software dependencies.

1. Environment

The system is built on an isolated virtual environment (venv) to ensure dependency abstraction and eliminate version conflicts with global system libraries:

- Core Runtime: **Python 3.11.x (64-bit)**
- Execution Interface: **Windows Command Prompt (CMD)**

2. Dependencies

Packages used in v0.1:

tensorflow	Core Deep Learning framework used to compile structure and execute the LSTM network
yfinance	Automated financial data retrieval tool used to fetch historical structural data for CBA.AX via the Yahoo Finance API
scikit-learn	Provides data preprocessing mechanics (specifically utilized for the MinMaxScaler engine to normalize array bounds between 0 and 1)
pandas	Sequential series concatenation, index cleaning and CSV file extraction
numpy	Low-level vectorized array manipulation, matrix dimension reshaping and numeric tensor optimization
matplotlib	Render visual evaluations

3. Deployment

To reconstruct this environment layout from scratch, run the following sequential commands within the target file directory: **(make sure you have installed Python 3.11)**

```
# Initialize the Python 3.11 Isolation layer
py -3.11 -m venv venv

# Activate the localized scripts block (Windows)
venv\Scripts\activate.bat

# Synchronize deployment prerequisites
pip install -r requirements.txt
```

Stock Prediction version 1

1. Existing Architecture

- **Asynchronous Data Acquisition:**
  Live daily historical financial data for the Commonwealth Bank of Australia (CBA.AX) is compiled via the Yahoo Finance API interface

```
# DATA_SOURCE = "yahoo"
COMPANY = 'CBA.AX'

TRAIN_START = '2020-01-01'     # Start date to read
TRAIN_END = '2023-08-01'       # End date to read

# data = web.DataReader(COMPANY, DATA_SOURCE, TRAIN_

import yfinance as yf

# Get the data for the stock AAPL
data = yf.download(COMPANY,TRAIN_START,TRAIN_END)

```

Figure 1.1 - Asynchronous Data Acquisition

- **Feature Normalization:**
  Target parameters are scaled into [0,1] using a bounded MinMaxScaler to constrain continuous raw financial values

```
#------------------------------------------------------------------------------
PRICE_VALUE = "Close"

scaler = MinMaxScaler(feature_range=(0, 1))
# Note that, by default, feature_range=(0, 1). Thus, if you want a different
# feature_range (min,max) then you'll need to specify it here
scaled_data = scaler.fit_transform(data[PRICE_VALUE].values.reshape(-1, 1))
# Flatten and normalise the data
# First, we reshape a 1D array(n) to 2D array(n,1)
```

Figure 1.2 - Feature Normalization

- **Temporal Sliding Windows:**
  Independent inputs are generated by blocks of 60-day sequences (PREDICTION_DAYS = 60) to map multi-day sequence contexts into individual training samples

```
# Number of days to look back to base the prediction
PREDICTION_DAYS = 60 # Original

# To store the training data
x_train = []
y_train = []

scaled_data = scaled_data[:,0] # Turn the 2D array back to
# Prepare the data
for x in range(PREDICTION_DAYS, len(scaled_data)):
    x_train.append(scaled_data[x-PREDICTION_DAYS:x])
    y_train.append(scaled_data[x])

# Convert them into an array
x_train, y_train = np.array(x_train), np.array(y_train)
# Now, x_train is a 2D array(p,q) where p = len(scaled_dat
# and q = PREDICTION_DAYS; while y_train is a 1D array(p)
```

Figure 1.3 - Temporal Sliding Windows

- **The Multi-Layer Filter:**
  Instead of passing the stock data through just one layer of neurons, the model stacks 3 LSTM layers on top of each other:
  - **Layer 1:** Looks at raw, day-to-day text and catches immediate details
  - **Layer 2:** Takes the notes from Layer 1 and looks for weekly trends
  - **Layer 3:** Takes the summaries from Layer 2 and identifies the massive, long-term market trends over months or years.

  50 hidden nodes each layer are equivalent to 50 distinct "thinking channels" to look at different patterns simultaneously

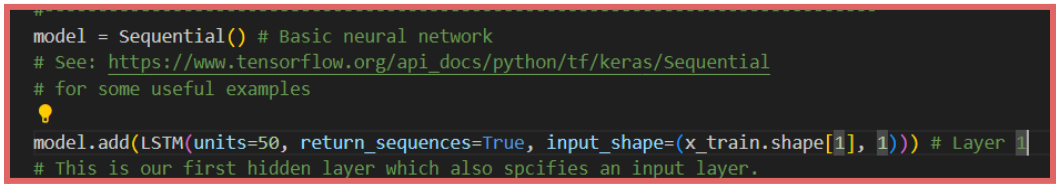


Figure 1.4 - LSTM Layer 1 with 50 nodes

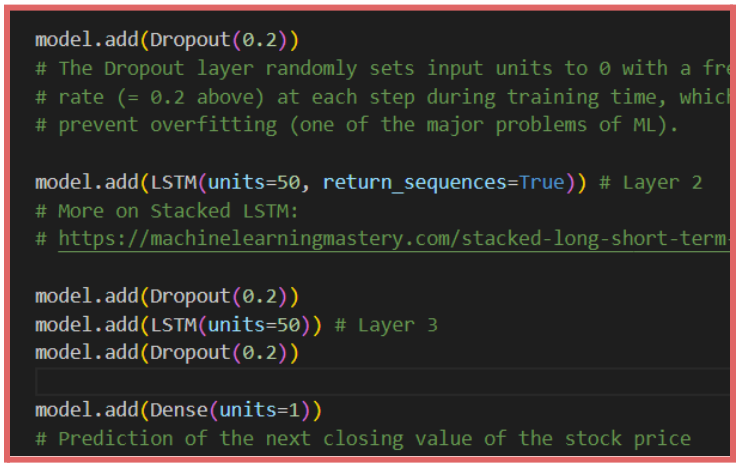


Figure 1.5 - LSTM Layer 2 and Layer 3 with 50 nodes each

- **Regularization Overlay via Dropout:**
  - A major trap in machine learning is overfitting (memorization). Because stock data is incredibly noisy, a complex model will try to perfectly memorize every historical bump in the chart instead of learning the actual underlying patterns
  - Dropout mechanism: At every single training step, the computer forcefully "knocks out" 20% of the neurons completely at random (10 out of the 50 nodes)
  - Real-life Example: Imagine a high-school study group of 5 students preparing for a final exam. If they always study together, one ultra-smart student might do all the heavy lifting, while the other 4 just copy their answers. They memorize the practice test perfectly, but if the actual exam has different questions, they fail.
  - Figure 1.4 and 1.5 shows that after every time a LSTM layer is added, a Dropout(0.2) will be called

2. Architectural Flaws

- **The Temporal Lag Illusion:**
  Instead of actually figuring out where the stock price is going tomorrow, the model plays it safe to avoid getting a bad score (high mathematical error). It simply looks at today's price and copies it as tomorrow's prediction, adding a tiny random adjustment
- **Data Scaling Volatility:**
  - Neural networks work best when all their input data is compressed into a tight, standardized scale between 0 and 1
  - If the stock price skyrockets to 180$ in 2024, the mathematical scaler turns that into a number like 1.8. Because the LSTM was only trained to handle numbers between 0 and 1, this unexpected "out-of-bounds" input completely confuses the network's math, leading to wild, inaccurate guesses
- **Market Tunnel Vision:**
  - The model is trying to predict the incredibly chaotic global stock market by looking at just 1 single column of data: The historical closing price
  - Stock prices don't move in a vacuum. They react to:
    - Trading Volume (how many shares are being bought or sold)
    - Technical Indicators (like RSI (Relative Strength Index) or MACD (Moving Average Convergence Divergence))
    - Macroeconomics (interest rates, inflation)
    - Public Sentiment (breaking news, viral tweets)

    Version v0.1 ignores all of them
- **Volatile Resource Persistence:**
  - The script has no long-term memory outside of the immediate moment it is running. It doesn't save anything to your computer's hard drive
  - Every time the script runs, it forces the computer to re-download all the data from the internet (wasting time and API limits) and spends a minute running 25 epochs to train the model from scratch. If the program is closed, that trained model disappears forever

3. Results Validation

Model execution was confirmed via progressive loss optimization over 25 epochs. As seen on the left side of Figure 3, the training environment successfully orchestrated 27 steps per epoch, with a step latency averaging 27ms to 35ms. The internal cost function tracked a Mean Squared Error (MSE) loss of 0.0055 by Epoch 24. This consistent downward convergence confirms proper mathematical gradient descent.

The actual and predicted pricing metrics were plotted using matplotlib (shown on the right side of Figure 3). The output chart maps out approximately 230 trading days.

While the green line (Predicted CBA.AX Price) captures the macro-level upward trend of the black line (Actual CBA.AX Price) between time intervals 75 and 200, it clearly demonstrates the Temporal Lag Illusion and Data Scaling Volatility inherent to version v0.1. As the actual stock price climbs past $115, the model's predictions suffer from compressed weight boundaries, heavily underestimating the peak asset value by capping out around $108.

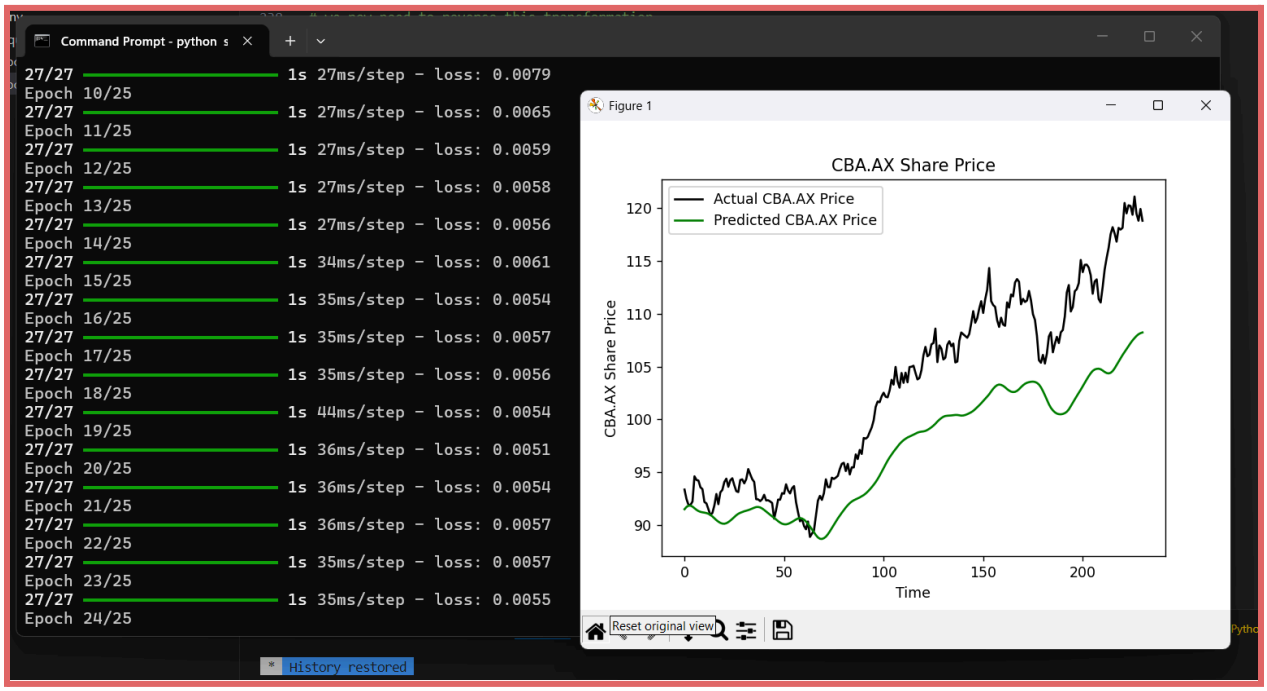


Figure 3.1 - Results

Comparison between v0.1 and P1

	v0.1	P1
Feature Input	Univariate (Close price only)	Multivariate (Multiple pricing and volume vectors)
Temporal Forecasting Horizon	1 Day forward	15 Days forward
Historical Data Scaling Scope	Fixed Window (around 230 Trading days)	Multiple Decades (1998 - 2026)
Mathematical Cost Function	MSE (Mean Squared Error)	Huber Loss
Output	Raw Abstract Decimal Loss Score	Real dollar MAE (Mean Absolute Error)

Table - Brief Comparison between v0.1 and P1

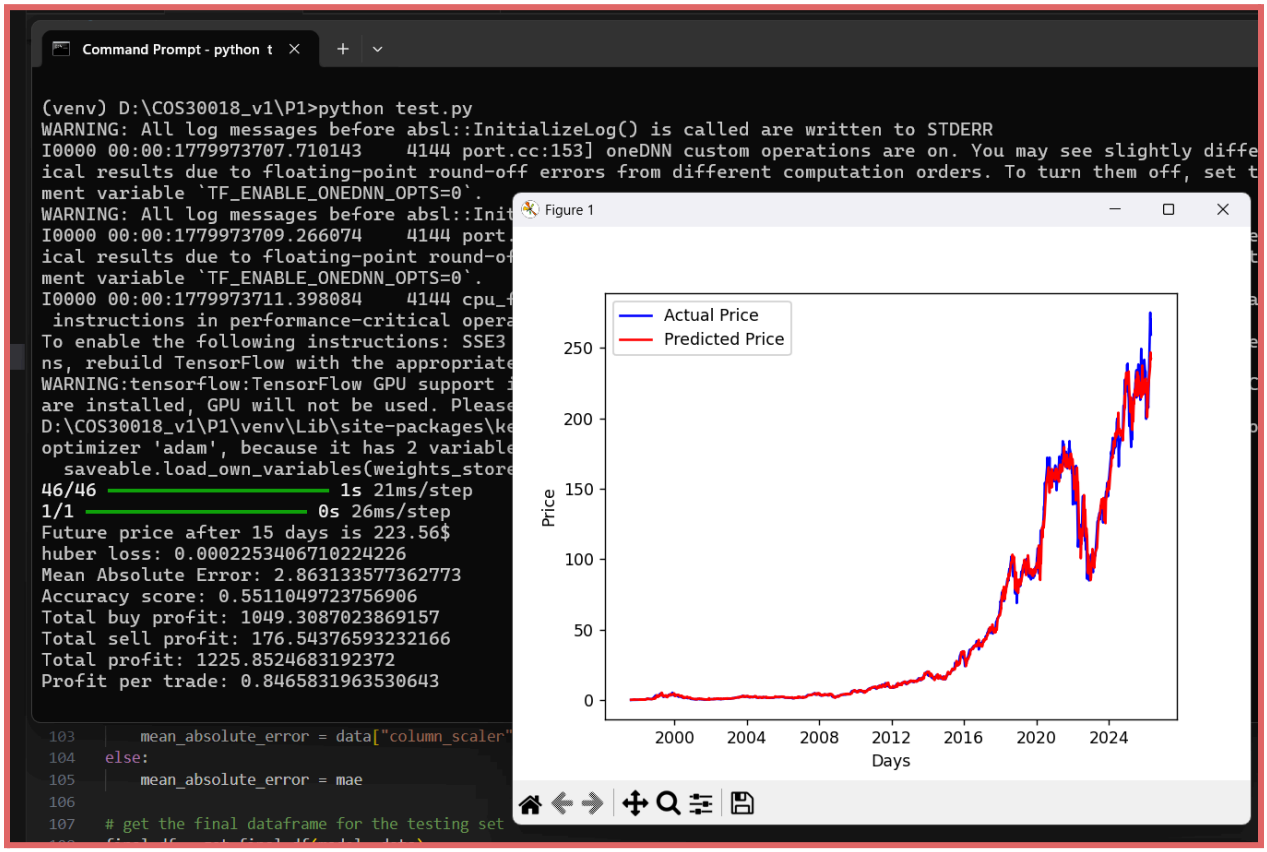
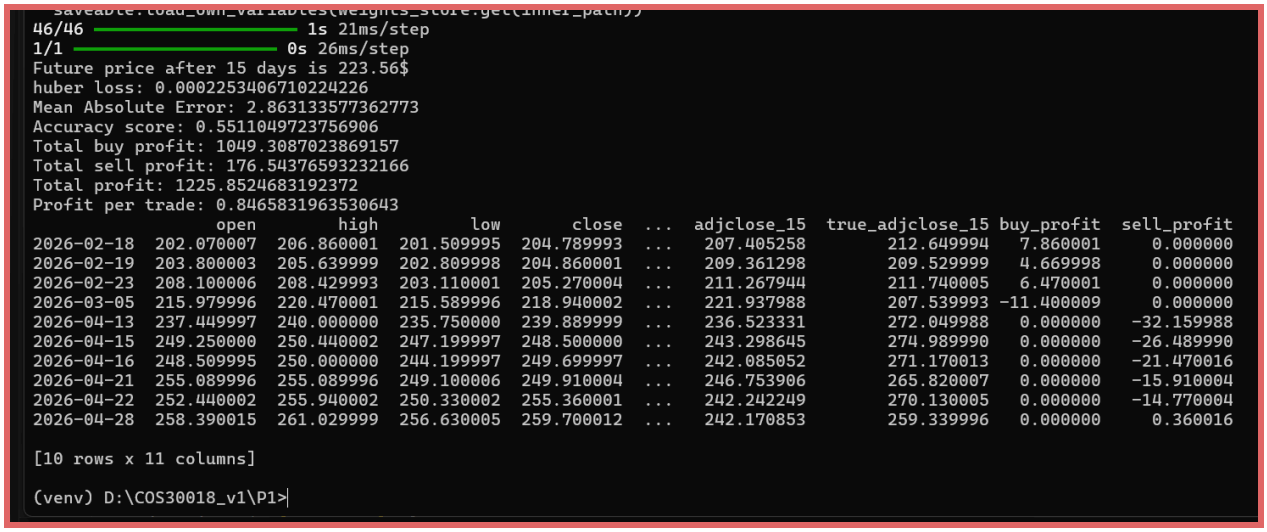


Figure 4.1 - Graph and CMD Outputs before closing Graph display



```
  saveable.load_own_variables(weights_store.get(inner_path))
46/46 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step
Future price after 15 days is 223.56$
huber loss: 0.0002253406710224226
Mean Absolute Error: 2.863133577362773
Accuracy score: 0.5511049723756906
Total buy profit: 1049.3087023869157
Total sell profit: 176.54376593232166
Total profit: 1225.8524683192372
Profit per trade: 0.8465831963530643
                open        high         low       close  ...  adjclose_15  true_adjclose_15 buy_profit  sell_profit
2026-02-18  202.070007  206.860001  201.509995  204.789993  ...   207.405258        212.649994   7.860001     0.000000
2026-02-19  203.800003  205.639999  202.809998  204.860001  ...   209.361298        209.529999   4.669998     0.000000
2026-02-23  208.100006  208.429993  203.110001  205.270004  ...   211.267944        211.740005   6.470001     0.000000
2026-03-05  215.979996  220.470001  215.589996  218.940002  ...   221.937988        207.539993 -11.400009     0.000000
2026-04-13  237.449997  240.000000  235.750000  239.889999  ...   236.523331        272.049988   0.000000   -32.159988
2026-04-15  249.250000  250.440002  247.199997  248.500000  ...   243.298645        274.989990   0.000000   -26.489990
2026-04-16  248.509995  250.000000  244.199997  249.699997  ...   242.085052        271.170013   0.000000   -21.470016
2026-04-21  255.089996  255.089996  249.100006  249.910004  ...   246.753906        265.820007   0.000000   -15.910004
2026-04-22  252.440002  255.940002  250.330002  255.360001  ...   242.242249        270.130005   0.000000   -14.770004
2026-04-28  258.390015  261.029999  256.630005  259.700012  ...   242.170853        259.339996   0.000000     0.360016

[10 rows x 11 columns]

(venv) D:\COS30018_v1\P1>
```

Figure 4.2 - Graph and CMD Outputs after closing Graph display

1. Temporal Forecasting Horizon

- **v0.1:** This model only looks 1 day ahead. To keep its mathematical error score low, it plays it safe: It simply copies today's price as its guess for tomorrow. This creates a Temporal Lag Illusion, the green line looks like a perfect match, but it is actually just trailing 24 hours behind reality, making it useless for real trading.

- **P1:** P1 successfully breaks the lag trap by looking 15 days into the future. It predicts a structural trend two weeks out, tracking the true momentum of the stock rather than just copying yesterday's homework.

2. Historical Data Scaling Scope

- **v0.1:** This model looks into a tiny time window of about 230 days (2023 to 2024). Furthermore, it exhibits Data Scaling Volatility: when the real stock price climbs to an all-time high over $115, the model's math panics because it has never seen numbers that large. Its predictions hit a ceiling and flatten out around $108.
  (See the right side of Figure 3.1)

- **P1:** This model easily handles a multi-decade timeline spanning from 1998 all the way to 2026. It smoothly follows massive, long-term market cycles, including huge growth periods and deep economic crashes, without its math breaking down or overflowing as the stock price climbs from $0 to over $250.
  (See the right side of Figure 4.1)

3. Mathematical Cost Function

- **v0.1:** Uses standard Mean Squared Error. If a stock has a sudden, random price spike due to a single news headline, v0.1 overreacts to that outlier and completely messes up the network's internal learning weights.
  (See the left side of Figure 3.1)

- **P1:** Records a Huber Loss of 0.000225. Huber loss acts like a mathematical shock absorber: it pays close attention to normal trends but ignores wild, one-off market spikes. P1 also translates its abstract errors into a real-dollar Mean Absolute Error (MAE) of $2.86, meaning that across decades of data, its price guesses are off by an average of only $2.86.
  (See Figure 4.2 or the left side of Figure 4.1)

4. Actionable Real World Trading Utility

- **v0.1:** This model does not trade. It simply draws a line on a screen.

- **P1:** This includes a built-in algorithmic simulation engine that converts math into real trading decisions:

    - Directional Accuracy (0.551): It guesses whether the market trend is moving UP or DOWN correctly 55.1% of the time.
      (See the Accuracy Score of Figure 4.1 or 4.2)

    - Simulated Profit ($1,225.85): By executing automated buy and sell positions during the test run, it banks a Total Buy Profit of $1,049.30, handles short positions, subtracts losing trades, and finishes with a clear, verified Total Profit of $1,225.85 (averaging about $0.85 of profit per trade).
      (See the Profit lines of Figure 4.1 or 4.2)